

Agentic BI Platform — Data Dictionary

This document describes every table in the platform’s PostgreSQL schema. It is the authoritative reference for table/column meaning and is also used as a RAG source document so the agent can answer questions like “what does active customer mean” or “where do I find return reasons.”

Two data provenances exist in this schema and are marked accordingly:

- **[REAL]** — derived from the public Olist Brazilian E-Commerce dataset (Kaggle, CC BY-NC-SA)
- **[SYNTHETIC]** — generated by this project to support business scenarios not present in Olist

1. customers [REAL]

One row per unique real-world person, keyed on Olist’s `customer_unique_id` (not `customer_id`, which is order-scoped — see `customer_crosswalk` below).

Column	Type	Constraints	Description
customer_unique_id	TEXT	PK	Stable identifier for one real person across all their orders
customer_zip_code_prefix	TEXT		First 5 digits of the customer’s zip code
customer_city	TEXT		Customer’s city
customer_state	TEXT		2-letter Brazilian state code (e.g. <code>SP</code>)

Note: Olist issues a brand-new `customer_id` for every order, even for repeat shoppers. Using it directly as a customer key overcounts customers. This table collapses to one row per real person via `customer_unique_id`.

2. customer_crosswalk [REAL]

Maps Olist’s order-scoped `customer_id` back to the real person. Exists purely for lineage/audit purposes — everyday analytical queries should not need it, since `orders` references `customer_id` directly and joins through this table to reach `customers`.

Column	Type	Constraints	Description
customer_id	TEXT	PK	Olist’s original per-order customer identifier
customer_unique_id	TEXT	FK → customers	The real person this order-level id belongs to

Query pattern: any customer-level analysis (repeat purchase rate, CLV, active customer counts) must join

orders → customer_crosswalk → customers and group by customer_unique_id, never by orders.customer_id directly.

3. orders [REAL]

One row per order.

Column	Type	Constraints	Description
order_id	TEXT	PK	Unique order identifier
customer_id	TEXT	FK → customer_crosswalk	Order-scoped customer id (see crosswalk note above)
order_status	TEXT		e.g. delivered, shipped, canceled, processing
order_purchase_timestamp	TIMESTAMP		When the order was placed
order_approved_at	TIMESTAMP	NULLABLE	When payment was approved; NULL if never approved (e.g. canceled)
order_delivered_carrier_date	TIMESTAMP	NULLABLE	Handoff to carrier; NULL if not yet shipped
order_delivered_customer_date	TIMESTAMP	NULLABLE	Actual delivery date; NULL if not delivered
order_estimated_delivery_date	TIMESTAMP		Estimated delivery date shown to customer at purchase

Note: Nullable delivery/approval timestamps are expected, not data-quality errors — orders that are canceled or still in progress will never populate these columns. Filter explicitly in time-to-delivery analyses.

4. order_items [REAL]

Bridge table resolving the many-to-many relationship between orders and products (one order can contain many products; one product appears in many orders). Also links each line item to the seller who fulfilled it.

Column	Type	Constraints	Description
order_id	TEXT	PK (composite), FK → orders	Order this line item belongs to
order_item_id	INTEGER	PK (composite)	Sequence number within the order (1, 2, 3...) — not globally unique alone
product_id	TEXT	FK → products	Product purchased

seller_id	TEXT	FK → sellers	Seller who fulfilled this item
shipping_limit_date	TIMESTAMP		Seller's shipping deadline for this item
price	NUMERIC(10,2)		Item price (excludes freight)
freight_value	NUMERIC(10,2)		Shipping cost for this item

Note: Primary key is composite (`order_id` , `order_item_id`) because `order_item_id` restarts at 1 for every order. Any table referencing a specific line item (e.g. `returns`) must carry both columns as its FK.

5. order_payments [REAL]

One or more payment records per order — an order can be paid via multiple methods (e.g. partial voucher + credit card).

Column	Type	Constraints	Description
order_id	TEXT	PK (composite), FK → orders	Order being paid for
payment_sequential	INTEGER	PK (composite)	Sequence of this payment if split across methods
payment_type	TEXT		e.g. <code>credit_card</code> , <code>boleto</code> , <code>voucher</code> , <code>debit_card</code>
payment_installments	INTEGER		Number of installments
payment_value	NUMERIC(10,2)		Amount paid in this payment record

6. order_reviews [REAL]

Customer review left after an order, typically one per order.

Column	Type	Constraints	Description
review_id	TEXT	PK	Unique review identifier
order_id	TEXT	FK → orders	Order being reviewed
review_score	SMALLINT		1–5 star rating
review_comment_title	TEXT	NULLABLE	Optional short title
review_comment_message	TEXT	NULLABLE	Optional free-text comment
review_creation_date	TIMESTAMP		When the review request was sent

review_answer_timestamp	TIMESTAMP		When the customer submitted the review
-------------------------	-----------	--	--

7. products [REAL]

Column	Type	Constraints	Description
product_id	TEXT	PK	Unique product identifier
product_category_name	TEXT	FK → product_category_name_translation, NULLABLE	Category in Portuguese; some products are uncategorized in source data
product_name_length	INTEGER		Character length of product name (renamed from Olist's <code>product_name_lenght</code>)
product_description_length	INTEGER		Character length of product description (renamed from Olist's <code>product_description_lenght</code>)
product_photos_qty	INTEGER		Number of photos listed
product_weight_g	NUMERIC		Weight in grams
product_length_cm	NUMERIC		
product_height_cm	NUMERIC		
product_width_cm	NUMERIC		

Note: `product_name_length` / `product_description_length` are corrected spellings of Olist's original `..._lenght` columns. Anyone cross-referencing Olist's public documentation should be aware of this rename.

8. product_category_name_translation [REAL]

Lookup table translating Portuguese category names to English.

Column	Type	Constraints	Description
product_category_name	TEXT	PK	Category name in Portuguese
product_category_name_english	TEXT		English translation

9. sellers [REAL + SYNTHETIC FK]

--	--	--	--

Column	Type	Constraints	Description
seller_id	TEXT	PK	Unique seller identifier
seller_zip_code_prefix	TEXT		First 5 digits of seller's zip code
seller_city	TEXT		
seller_state	TEXT		2-letter state code
employee_id	TEXT	FK → employees, NULLABLE	Account manager assigned to this seller (synthetic)

Note: A seller's region is *not* stored directly — it is derived through `employee_id → employees.region_id → regions`, to avoid storing the same fact in two places.

10. regions [SYNTHETIC]

Sales territory groupings, coarser than individual states, used for region-level reporting.

Column	Type	Constraints	Description
region_id	INTEGER	PK	Unique region identifier
region_name	TEXT		e.g. "Southeast", "Northeast"

11. state_region_map [SYNTHETIC]

Lookup mapping each Brazilian state code to its sales region.

Column	Type	Constraints	Description
state_code	TEXT	PK	2-letter state code
region_id	INTEGER	FK → regions	Region this state belongs to

12. employees [SYNTHETIC]

Account managers responsible for seller relationships.

Column	Type	Constraints	Description
employee_id	TEXT	PK	Unique employee identifier
employee_name	TEXT		
role	TEXT		e.g. "Account Manager"

region_id	INTEGER	FK → regions	Region this employee operates in
hire_date	DATE		

13. inventory [SYNTHETIC]

Stock level per product. Modeled 1:1 with `products` (single-warehouse simplification — no warehouse/location dimension in this version).

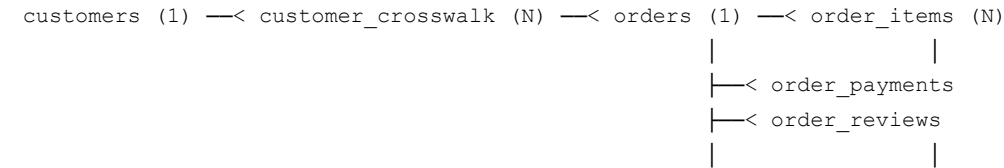
Column	Type	Constraints	Description
product_id	TEXT	PK, FK → products	Product being tracked
stock_quantity	INTEGER		Current units in stock
restock_date	DATE		Date of most recent restock

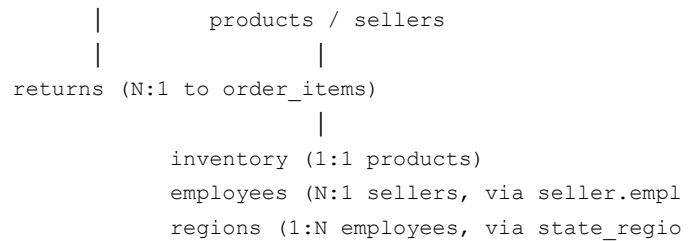
14. returns [SYNTHETIC]

Return of a specific order line item — not the whole order, since a customer may return only some items from a multi-item order.

Column	Type	Constraints	Description
return_id	TEXT	PK	Unique return identifier
order_id	TEXT	FK → order_items (composite, part 1)	
order_item_id	INTEGER	FK → order_items (composite, part 2)	
return_reason	TEXT		e.g. “damaged”, “wrong item”, “changed mind”
return_date	DATE		Must be ≥ the order’s purchase date
refund_amount	NUMERIC(10,2)		Amount refunded

Key Relationships Summary





Known Data Quirks (Read Before Writing Queries)

1. **Customer identity:** always join through `customer_crosswalk` to reach `customers.customer_unique_id` for any customer-level metric. Never treat `orders.customer_id` as a person-level identifier.
2. **Composite keys:** `order_items`, `order_payments`, and `returns` (via `order_items`) require two-column joins, not single-column.
3. **Nullable timestamps** in `orders` reflect real order lifecycle states (canceled/in-progress orders), not missing data errors.
4. **Region is derived, not stored** on `sellers` or `orders` directly — always resolved through the employee → region chain (sellers) or the state → region map (customers).
5. **Geolocation** (Olist's `geolocation` dataset) is not included in this schema as a strict foreign-key relationship — it is a loose lookup by non-unique zip-code prefix and will be handled separately if/when needed.